

Group Project Code

Madeline, Maisie, Audrey

2026-05-11

1. Set up

Packages

```
# general use
library(tidyverse)
library(janitor)

# file organization
library(here)

# Boxplot Visualization

library(hrbrthemes)

library(viridis)

# text label instead of legend
library(cowplot)

# Ridgeline Plot
#install.packages("ggribes")
library(ggribes)
#install.packages("ggplot2")
library(ggplot2)

# Color Palette
library(NatParksPalettes)
library(wesanderson)
```

```
# Use ragg_png device for better font rendering and system font support
knitr::opts_chunk$set(dev = "ragg_png")
```

Data

```
# weather data from NOAA weather station
weather <- read_csv(here("data", "NOAA-weather-data.csv"))

# water quality survey metadata
metadata <- read_csv(here("data", "NCOS_YSI_Water_Quality_Monitoring_0.csv"))

# water quality parameter measurements
water_parameters <- read_csv(here("data", "YSI_Data_Begin_1.csv"))
```

Code

```
weather_clean <- weather |>
  # made column names all lowercase
  clean_names() |>
  # puts date in proper format and makes sure its understood as date (not as chr)
  mutate(date = mdy(date)) |>
  # keeping only these 4 columns of interest
  select(date, prcp, tmax, tmin) |>
  # creating new columns for month and year
  # extracting month from the date (needs to be in the standard date format for this function)
  mutate(month = month(date),
         year = year(date)) |>
  mutate(wy = case_when(
    # when month os october or later, assign a value of year +1
    # in the water ywar for next year
    month >= 10 ~ year + 1,
    # keeping all months < 10 the same (sept or earlier), assigning value of existing year
    .default = year
  ))
```

```
metadata_clean <- metadata |>
  # col names easier to work with (all lowercase w/ _)
```

```

clean_names() |>
# "!" is the "not-operator", columns we DONT want to keep, excluding
select(!c(object_id, creator, editor)) |>
mutate(monitoring_date = mdy_hms(monitoring_date)) |>
rename(monitoring_datetime = monitoring_date) |>
mutate(month = month(monitoring_datetime),
       year = year(monitoring_datetime)) |>
# assigning water year
mutate(wy = case_when(
  month >= 10 ~ year + 1,
  .default = year )) |>
# creatig new column for site full name
mutate(site_full = case_when(
  site_name == "east_channel" ~ "East Channel",
  site_name == "phelps_bridge" ~ "Phelps Bridge",
  site_name == "venoco_bridge" ~ "Venoco Bridge",
  site_name == "other" ~ "Other"
)) |>
# filter out Other observations
filter(site_full != "Other")

```

```

# creating a new object from water parameters
water_parameters_clean <- water_parameters |>
# cleaning column names
clean_names() |>
# "!" not operator, excluding unneeded columns
select(!c(creator, editor)) |>
# renaming depth code to be elevation code to better represent methodology
rename(elevation_code = depth_code) |>
## Everything below came from visualizing the data first and defining what to fix/add
# fixing a mistake in the data
mutate(elevation_code = case_when(
  # anytime theres a 10 in elevation code, replace it with a 1
  elevation_code == 10 ~ 1,
  # everything else, keep the same
  .default = elevation_code)) |>
mutate(
  # turning column into a factor
  elevation_code = as_factor(elevation_code),
  # defining the correct order for the levels of the factor
  elevation_code = fct_relevel(elevation_code,
                               # write in the order you want

```

```
"0", "1", "2", "3", "4", "5", "6"))
```

2. Joining

water quality = metadata + water parameters

```
# full joining
water_quality <- full_join(
  # left data frame
  x = metadata_clean,
  # right data frame
  y = water_parameters_clean,
  # name the columns in each df that you want to join by (naming the shared column)
  by = c("global_id" = "parent_global_id")) |>
select(
  # information about the observation
  global_id, monitoring_datetime, time_of_wq_measurement, monitor_name, weather, site_name
  # water information
  water_surface_elevation, flow, notes,
  # measurement information
  elevation_code, do_mg_l, do_percent_sat, conductivity_u_s_cm, salinity_ppt, temperature_c
) |>
# taking the date out of this column, dont want the time
mutate(date = date(monitoring_datetime)) |>
# moving the new date column to a better location within the df
relocate(date, .after = monitoring_datetime) |>
# dont need this column anymore
select(!monitoring_datetime)
```

water quality weather = water quality + weather

```
# left join
water_quality_weather <- left_join(
  # left data frame
  x = water_quality,
  # right data frame
  y = weather_clean,
```

```
# joining these dfs by their date column
by = "date")
```

3. Wrangling

Dealing with Outliers in Salinity

```
# creating new obj
salinity_outlier_ids <- water_quality_weather |>
  # filtering to only include outlier values
  filter(salinity_ppt > 1410) |>
  # extracting the global id column as a vector usign pull
  pull(global_id)
```

Get precipitation per month (site independent variable)

```
# create new object from `water_quality_weather`
monthly_prcp <- water_quality_weather |>
  # only include elevation codes 0, 1, and 2
  filter(elevation_code %in% c("0", "1", "2")) |>
  # get rid of site names w/ "other"
  filter(!site_full == "NA") |>
  # removing any duplicate rows based on the combination of month, year, and wy
  distinct(month, year, prcp) |>
  # grouping by month and year
  group_by(month, year) |>
  # calculating the sum of prcp per month (not per site as it is a weather event and not site)
  summarise(sum_prcp = sum(prcp, na.rm = TRUE)) |>
  # ungrouping
  ungroup() |>
  # creating a date column to be read as a date
  mutate(date = lubridate::make_date(.data$year, .data$month, 1)) |>
  # creating levels to `month`, putting them in a logical order
  mutate(month = factor(month,
    levels = 1:12,
    labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"),
    ordered = TRUE))
```

`summarise()` has grouped output by 'month'. You can override using the
`.groups` argument.

Filtering Salinity & Summarizing Variables (for final viz)

```
salinity_filtered <- water_quality_weather |>
  # keeping only elevation codes 0, 1, and 2
  filter(elevation_code %in% c("0", "1", "2")) |>
  # filtering the column global_id to exclude the values that match those in `salinity_outli
  filter(!global_id %in% salinity_outlier_ids) |>
  # dropping the column for other_site_names
  select(!other_site_name) |>
  # get rid of site names w/ "other"
  filter(!site_name == "other") |>
  # setting the order of the levels (from lowest to highest avg salinity value)
  # helps the order of the visualizations
  mutate(site_full = fct_relevel(site_full,
                                "Phelps Bridge", "East Channel", "Venoco Bridge")) |>

  # taking out sites w/ NA
  filter(!site_full == "NA") |>
  # keeping only columns of interest (to reduce confusion)
  select(site_full, water_surface_elevation, elevation_code, salinity_ppt, temperature_c, mo
  # creating levels to `month`, putting them in a logical order
  mutate(month = factor(month,
                        levels = 1:12,
                        labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "S
                        ordered = TRUE)) |>

  # group by site full, month, and year
  group_by(site_full, month, year) |>
  # calculate mean salinity (per site per month)
  summarize(mean_salinity = mean(salinity_ppt, na.rm = TRUE),
            .groups = "drop")
```

Joining Monthly Precipitation (sum) w/ Monthly Salinity (avg.) [for final viz]

```
salinity_prpc <- salinity_filtered |>
  left_join(monthly_prpc, by = c("month", "year"))
```

Making data frame for exploratory figures

```
# create new object `exploratory_salinity` from `water_quality_weather`
exploratory_salinity <- water_quality_weather |>
  # only keep elevation codes 0, 1, 2, and 3 (keeping three just for visualization)
  filter(elevation_code %in% c("0", "1", "2", "3")) |>
  # filtering the column global_id to exclude the values that match those in `salinity_outlier_ids`
  filter(!global_id %in% salinity_outlier_ids) |>
  # dropping the column `other_site_names`
  select(!other_site_name) |>
  # filterign out site names that match "other"
  filter(!site_name == "other") |>
  # setting the order of the levels (from lowest tp highest avg salinity value)
  # helps the order of the visualizations
  mutate(site_full = fct_relevel(site_full,
                                "Phelps Bridge", "East Channel", "Venoco Bridge")) |>

# taking out sites w/ NA
filter(!site_full == "NA") |>
# keeping only columns of interest (to reduce confusion)
select(site_full, water_surface_elevation, elevation_code, salinity_ppt, temperature_c, month, year) |>
# group by site, elevation code, month, and year
group_by(site_full, elevation_code, month, year) |>
# calculate mean salinity
summarize(mean_salinity = mean(salinity_ppt, na.rm = TRUE),
           .groups = "drop") |>
# creating levels to `month`, putting them in a logical order
mutate(month = factor(month,
                      levels = 1:12,
                      labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"),
                      ordered = TRUE))
```

Water Surface Elevation

```
# create a new object `monthly_surface_elv` from `water_quality_weather`
monthly_surfcae_elv <- water_quality_weather |>
  # only include elevation codes 0, 1, and 2
  filter(elevation_code %in% c("0", "1", "2")) |>
  # Filter out site names matchign to "other"
  filter(!site_name == "other") |>
  # group by site, month, and year
```

```

group_by(site_full, month, year) |>
# calculate mean water surface elevation
summarise(avg_elv = mean(water_surface_elevation, na.rm = TRUE)) |>
# ungroup data frame
ungroup() |>
# create a date column to be read as a date
mutate(date = lubridate::make_date(.data$year, .data$month, 1)) |>
# creating levels to `month`, putting them in a logical order
mutate(month = factor(month,
                      levels = 1:12,
                      labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"),
                      ordered = TRUE))

```

`summarise()` has grouped output by 'site_full', 'month'. You can override using the `.groups` argument.

```

## Joining data frames
elv_viz <- monthly_surfae_elv |>
# joining by columns month, year, and site
left_join(salinity_prcp, by = c("month", "year", "site_full")) |>
# filtering out all NA average elevation values
filter(!avg_elv == "NaN")

```

4. Distribution Visualizations, Comparing Groups

Boxplot w/ Jitter - Distribution of Mean Salinity Values Across Sites

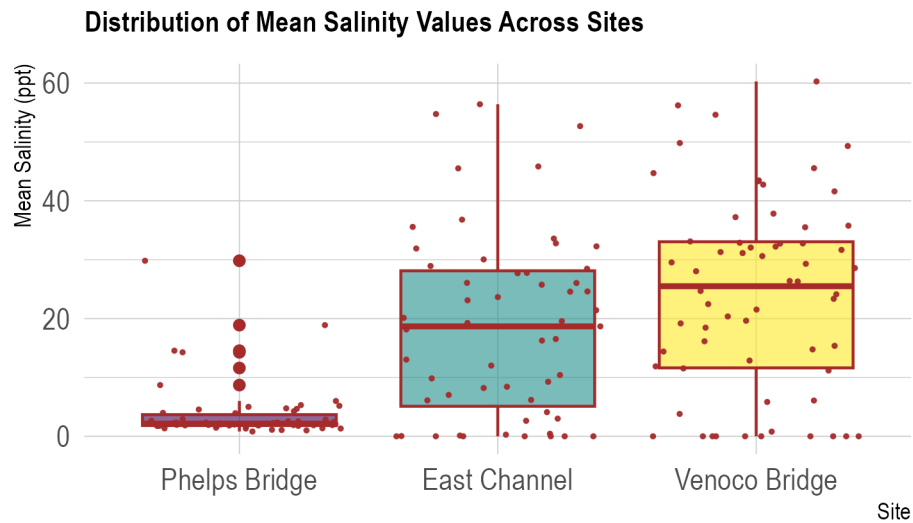
Comment from An from revisions

```

# Plot
salinity_prcp |>
ggplot(aes(x= site_full, y=mean_salinity, fill = site_full)) +
  geom_boxplot(color = "brown") +
  scale_fill_viridis(discrete = TRUE, alpha=0.6) +
  geom_jitter(color="brown", size=0.4, alpha=0.9) +
  theme_ipsum() +
  theme(
    legend.position="none",
    plot.title = element_text(size=11)
  )

```

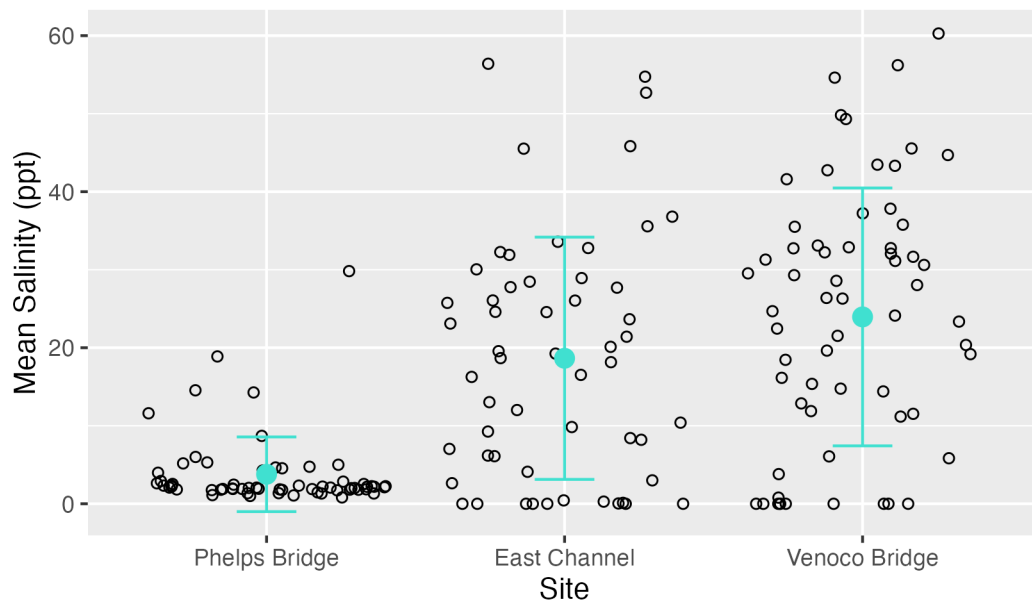
```
) +
  ggtitle("Distribution of Mean Salinity Values Across Sites") +
  xlab("Site") +
  ylab("Mean Salinity (ppt)")
```



Strip Chart w/ Standard Deviation - How Mean Salinity Differs Between Sites

```
ggplot(data = salinity_prpcp,
  mapping = aes(x = site_full,
    y = mean_salinity)) +
  geom_jitter(shape = 21) +
  stat_summary(fun = mean, geom = "point", size = 3, color = "turquoise") +
  stat_summary(fun.data = mean_sdl, fun.args = list(mult = 1), geom = "errorbar", width = 0.1) +
  labs(x = "Site",
    y = "Mean Salinity (ppt)",
    title = "How Mean Salinity Differs Between Sites")
```

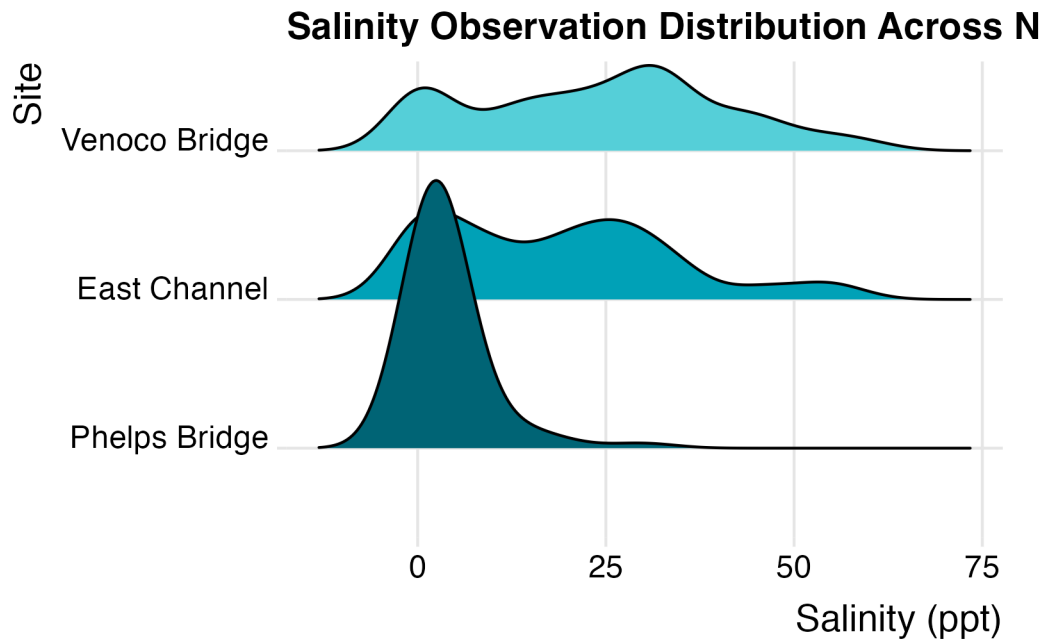
How Mean Salinity Differs Between Sites



Ridgeline Plot - Salinity Observation Distribution Across NCOS Sites

```
ggplot(salinity_prpc, aes(x = mean_salinity, y = site_full, fill = site_full)) +  
  geom_density_ridges() +  
  theme_ridges() +  
  theme(legend.position = "none") +  
  scale_fill_manual(values = natparks.pals("Banff")) +  
  labs(x = "Salinity (ppt)",  
       y = "Site",  
       title= "Salinity Observation Distribution Across NCOS Sites")
```

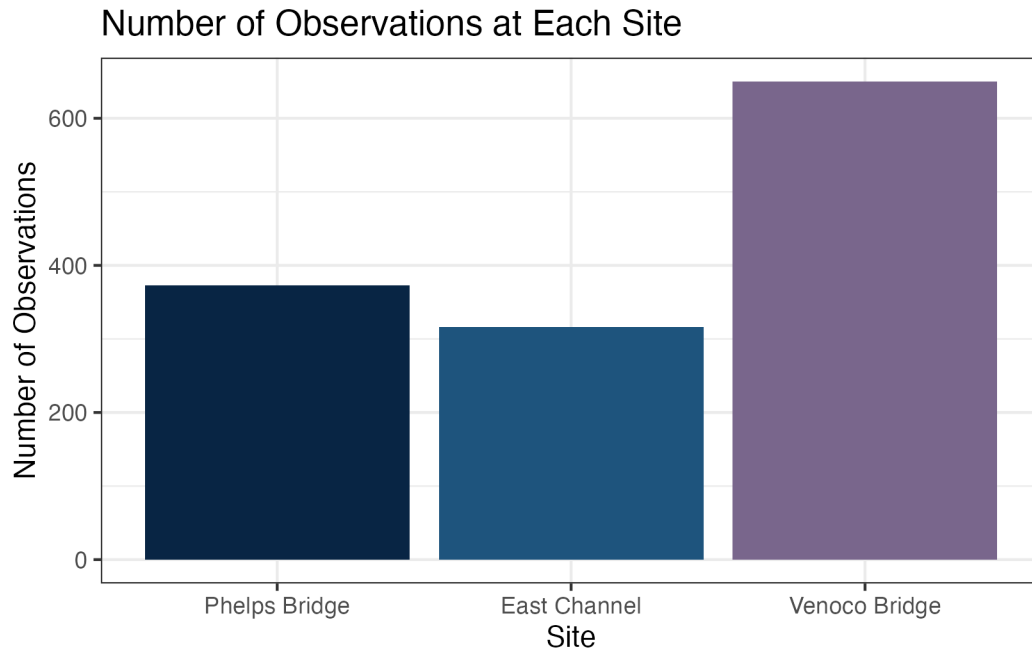
Picking joint bandwidth of 4.37



Bar Graphs (Distribution of Observations)

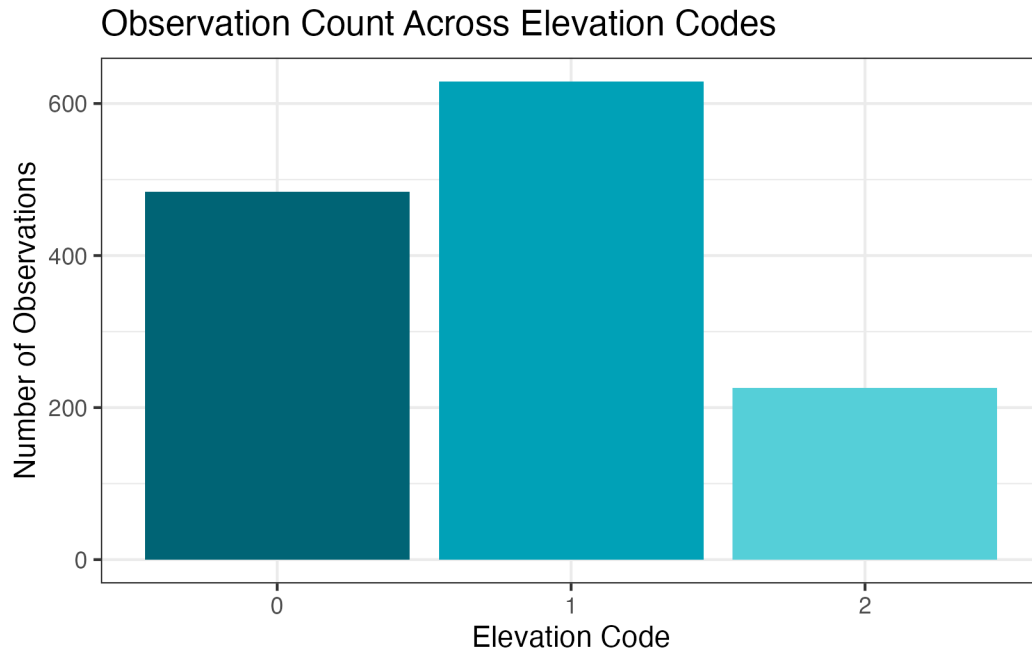
of Observations per Site

```
ggplot(data = water_quality_weather |>
  filter(!global_id %in% salinity_outlier_ids,
    site_name != "other",
    elevation_code %in% c("0", "1", "2")) |>
  mutate(site_full = fct_relevel(site_full,
    "Phelps Bridge", "East Channel", "Venoco Bridge")),
  mapping = aes(x = site_full)) +
scale_fill_manual(values = natparks.pals("Volcanoes")) +
geom_bar(aes(fill = site_full)) +
theme_bw() +
theme(legend.position = "none") +
labs(x = "Site",
  y = "Number of Observations",
  title = "Number of Observations at Each Site")
```



of Observations per Elevation Code

```
ggplot(data = water_quality_weather |>
  filter(!global_id %in% salinity_outlier_ids,
    site_name != "other",
    elevation_code %in% c("0", "1", "2")),
  mapping = aes(x = elevation_code)) +
geom_bar(aes(fill = elevation_code)) +
scale_fill_manual(values = natparks.pals("Banff")) +
theme_bw() +
theme(legend.position = "none") +
labs(x = "Elevation Code",
  y = "Number of Observations",
  title = "Observation Count Across Elevation Codes")
```



Precipitation Peaks

```
# creating a new object `precipitation_peaks` from `salinity_prpc`
precipitation_peaks <- salinity_prpc |>
  # including the dates of the top 4 monthly precipitation
  filter(date %in% c("2021-01-01", "2023-03-01", "2023-02-01", "2021-10-01")) |>
  # ungrouping data frame
  ungroup() |>
  # keeping these 4 distinct dates
  distinct(date, .keep_all = TRUE) |>
  # adding a label to each event
  mutate(label = c("Rain Event (5.19 in)", "Rain Event (2.06 in)", "Rain Event (1.80 in)", "Rain Event (1.80 in)"),
         # setting the y position for each event (dropping the 1.80 event lower for better view)
         label_y = c(4.5, 4.5, 3.5, 4.5))

precipitation_peaks_facet <- precipitation_peaks |>
  cross_join(tibble(site_full = unique(salinity_prpc$site_full))) |>
  mutate(label_y_2 = rep(c(40, 45, 40, 50), times = 3))
```

5. Figure Visualizing

Figure 1a. - Monthly Precipitation at NCOS (Time Series w/ Rain Event Lines)

```
# base layer: ggplot
# data: salinity_prctp
ggplot(data = salinity_prctp,
       # x axis: date
       mapping = aes(x = date,
                     # y axis: monthly precipitation sum
                     y = sum_prctp)) +
  # prcp displayed by columns (a suggestion made by An after work plan/proposal edits)
  geom_col(color = "darkblue") +
  # adding themeing
  theme_bw() +
  # x-axis breaks every 6 months (to reduce cognitive load)
  # Format date axis labels as abbreviated month and full year
  scale_x_date(date_breaks = "6 months", date_labels = "%b %Y") +
  # angling labels to 45 degrees and to be right-aligned
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  # expanding limits to make room for text
  scale_y_continuous(limits = c(0, 6)) +
  # labeling x and y axes
  labs(x = "Date",
       y = "Total Monthly Precipitation",
       # adding title
       title = "Total Monthly Precipitation at NCOS") +
  # removing legend, redundant
  theme(legend.position = "none") +
  # adding in lines for prcp events
  geom_vline(data = precipitation_peaks,
            # centering them on dates
            aes(xintercept = date,
               # coloring them by their established label
               color = label),
            # signified by dotted lines
            linetype = "dotted",
            # width of lines
            linewidth = 1.25) +
  # adding custom color palette
  scale_color_manual(values = wes_palette("AsteroidCity1")) +
```

```

# adding labels to each dotted line from the `precipitation_peaks` object
geom_text(data = precipitation_peaks,
  # x-axis location: date
  # y-axis location: set y value from earlier chunk
  # label = the defined label from earlier chunk
  # colro by label
  aes(x = date, y = label_y, label = label, color = label),
  # 45 degree angle for readability
  angle = 45,
  # slightly to right of vertical dotted line
  hjust = -0.1,
  # text size
  size = 3,
  # text bolding
  fontface = "bold")

```

Warning: Removed 3 rows containing missing values or values outside the scale range (`geom\_col()`).

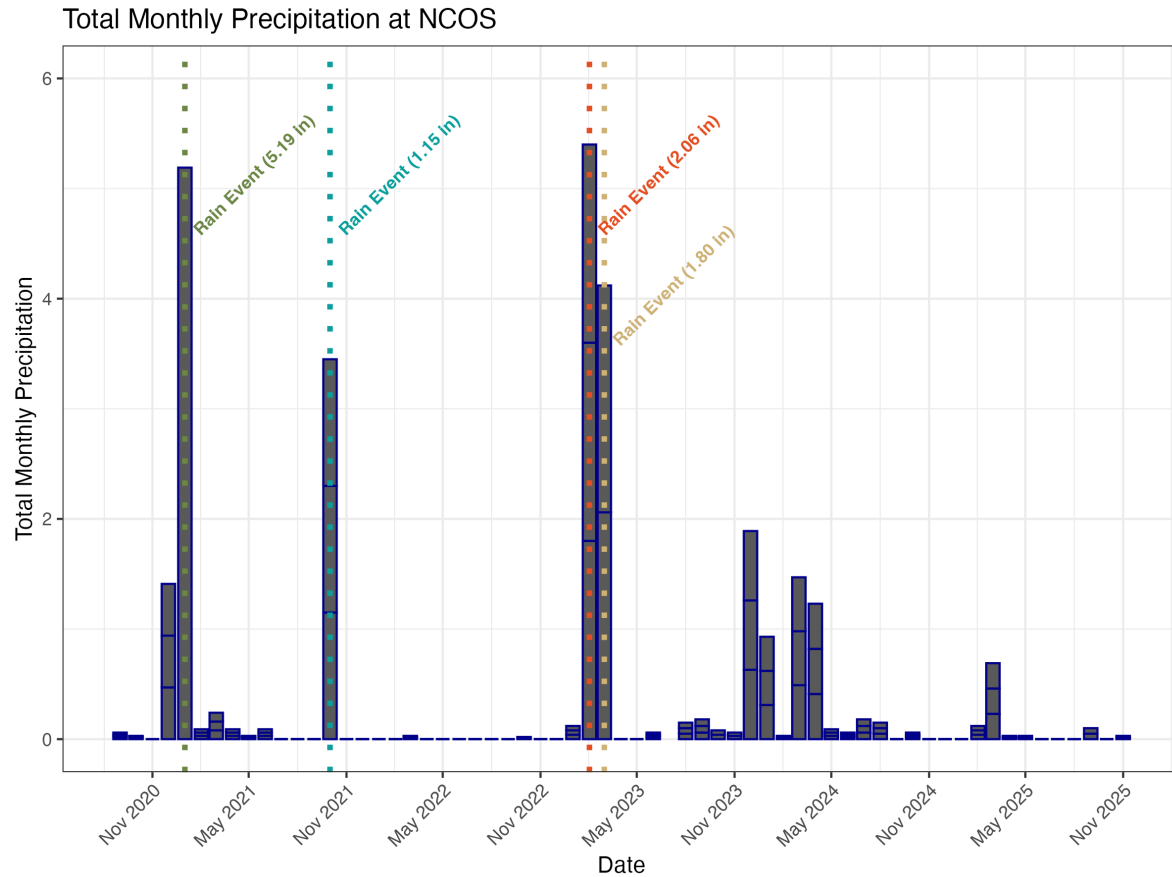


Figure 1b. Salinity Time Series w/ Prcp Highlights

```
# base layer ggplot
# data: salinity_prpc
ggplot(data = salinity_prpc,
  # x-axis: date
  mapping = aes(x = date,
    # y-axis: mean_salinity
    y = mean_salinity)) +
# column displaying mean salinity
geom_col(color = "grey") +
# separating points from sites into different plots
facet_wrap(~ site_full) +
# adding themeing
theme_bw() +
```

```

# x breaks by year to reduce cognitive load, labeling by abbreviated month and full year
scale_x_date(date_breaks = "12 months", date_labels = "%b %Y") +
# angling x axis labels to 45 degrees and to be right-aligned
theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
# labeling x and y axes
labs(x = "Date",
      y = "Monthly Average Salinity (ppt)",
      # adding title
      title = "Monthly Average NCOS Salinity",
      # adding legend title
      color = "Precipitation Events") +
# adding lines from the `precipitation_peaks_facet` object
geom_vline(data = precipitation_peaks_facet,
            # oriented on the date
            aes(xintercept = date,
                # different colors by label
                color = label),
            # long dash for better viz
            linetype = "longdash",
            # line width
            linewidth = 0.75) +
# custom colors
scale_color_manual(values = wes_palette("AsteroidCity1")) +
# legend at bottom for less visual clutter
theme(legend.position = "bottom")

```

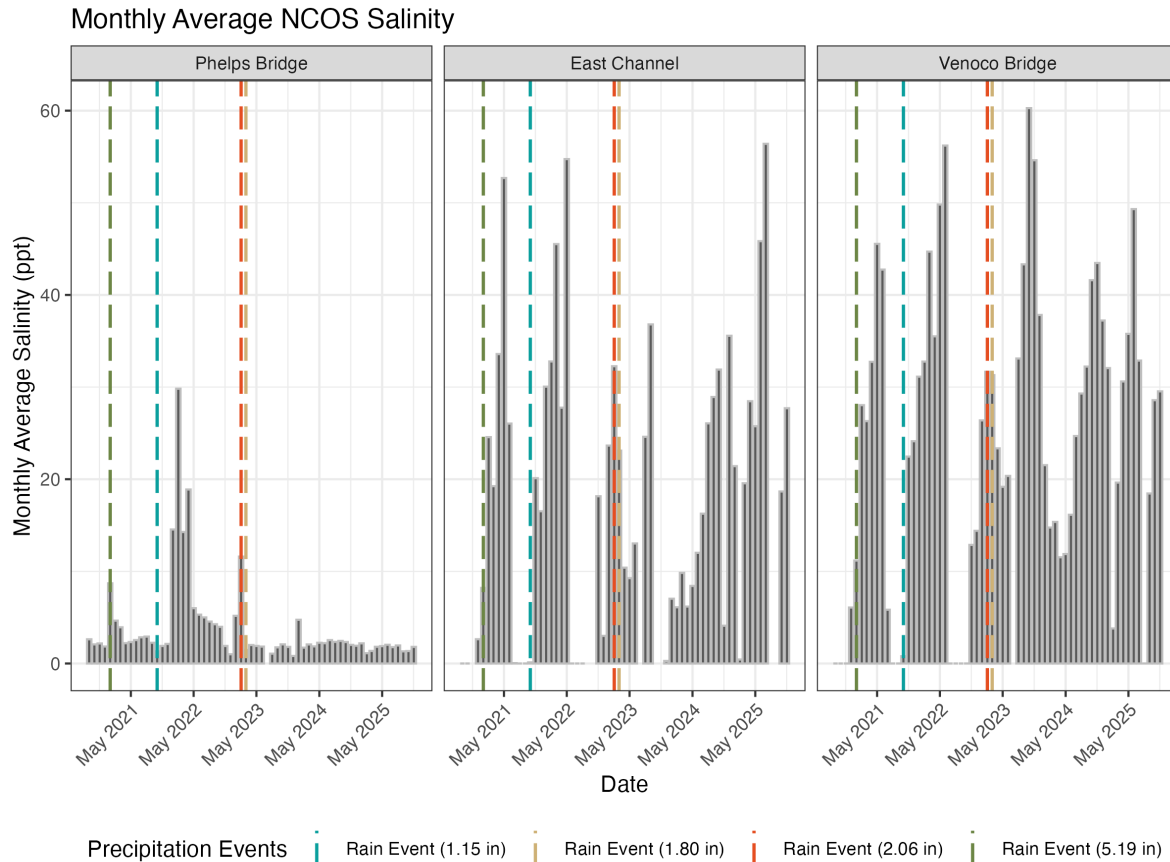


Figure 2. - How Salinity Relates to Water Surface Elevation (Venoco Bridge)

```
ggplot(data = elv_viz,
       mapping = aes(x = avg_elv,
                     y = mean_salinity)) +
  geom_point(color = "maroon") +
  facet_wrap(~ site_full) +
  scale_color_manual(values = wes_palette("GrandBudapest1")) +
  theme_bw() +
  labs(x = "Monthly Average Water Surface Elevation (m)",
       y = "Monthly Average Salinity (ppt)",
       title = "Figure 2. How Salinity Relates to Water Surface Elevation (Venoco Bridge)")
```

Figure 2. How Salinity Relates to Water Surface Elevation (Venoco Bridge)

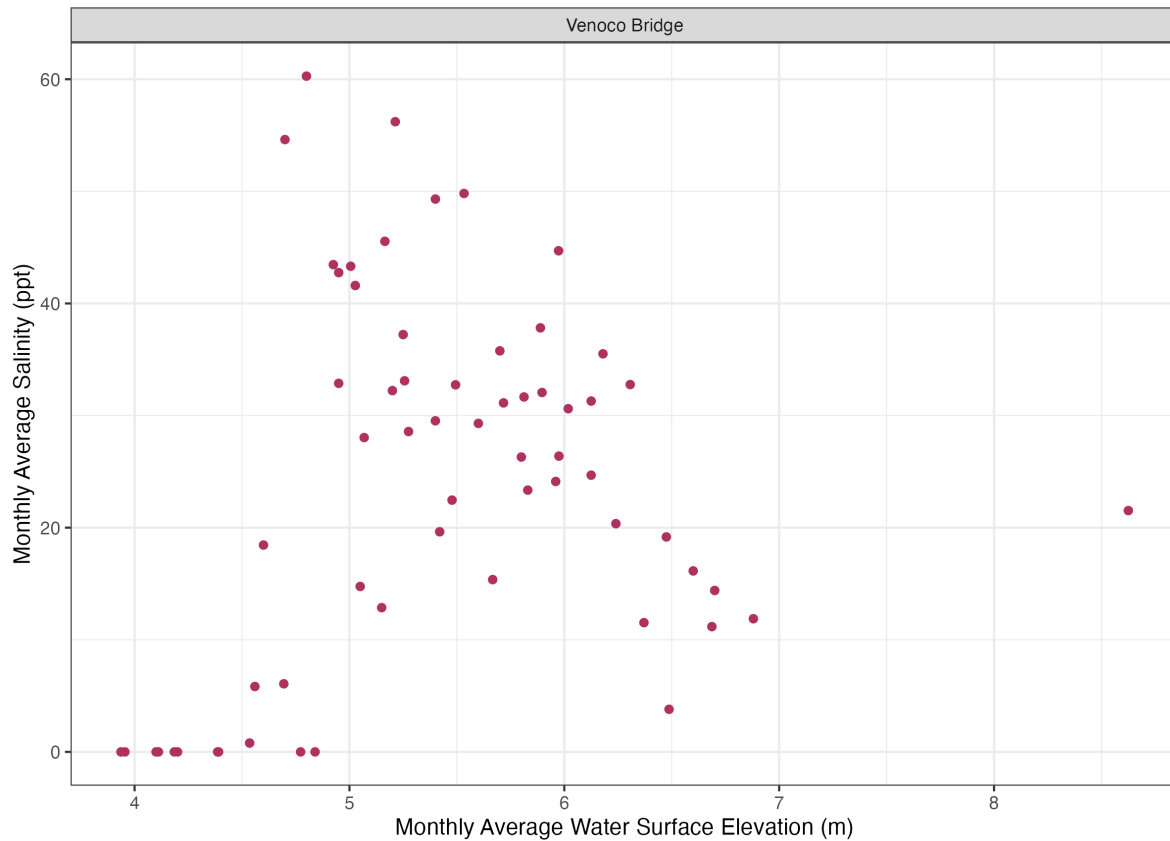


Figure 3. - How Salinity Varies Between Elevation Codes

```
ggplot(data = exploratory_salinity,
       mapping = aes(x = elevation_code,
                     y = mean_salinity)) +
  geom_point(shape = 21) +
  stat_summary(fun = median,
              geom = "point",
              color = "maroon") +
  stat_summary(fun = median,
              geom = "line",
              color = "maroon",
              group = 1) +
  labs(x = "Elevation Code",
       y = "Mean Salinity",
```

```

    title = "Figure 3. Mean Salinity Differences Between Elevation Codes") +
  facet_wrap(~ site_full) +
  theme_bw() +
  scale_x_discrete(breaks = c(0,1, 2, 3)) +
  scale_y_continuous(position = "right") +
  coord_flip()

```

Figure 3. Mean Salinity Differences Between Elevation Codes

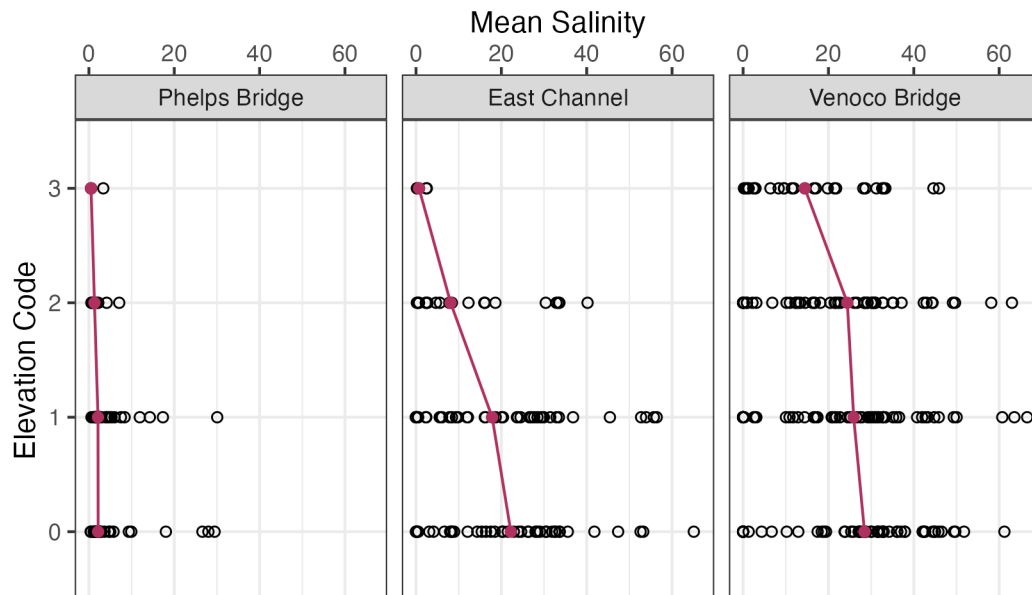


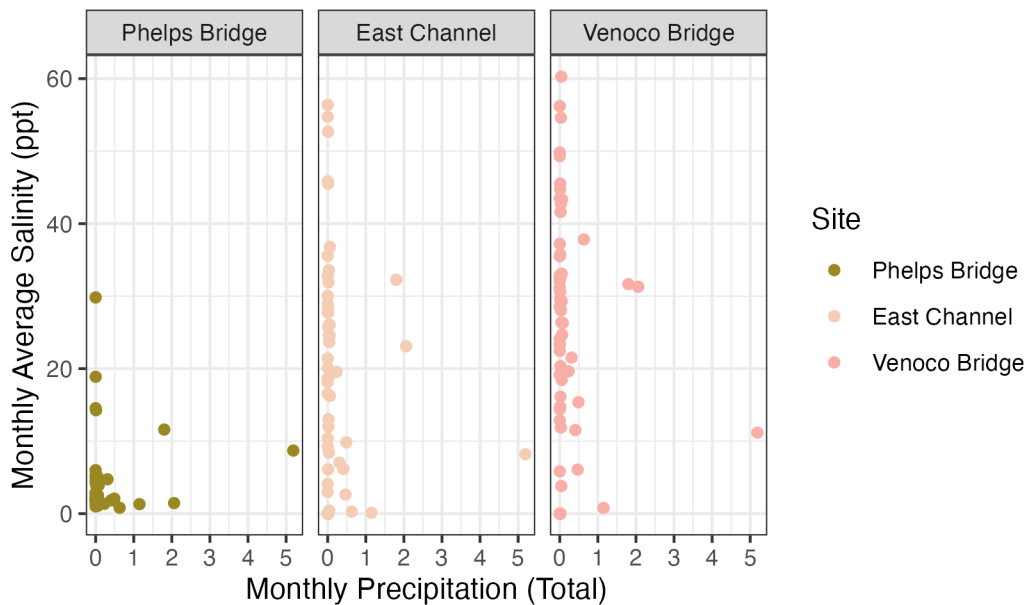
Figure 4. - How Precipitation Explains Salinity

```

ggplot(data = salinity_prctp,
  mapping = aes(x = sum_prctp,
    y = mean_salinity,
    color = site_full)) +
  geom_point() +
  facet_wrap(~ site_full) +
  scale_color_manual(values = wes_palette("Royal2")) +
  theme_bw() +
  labs(x = "Monthly Precipitation (Total)",
    y = "Monthly Average Salinity (ppt)",
    title = "Figure 4. How Precipitation Explains Salinity",
    color = "Site")

```

Figure 4. How Precipitation Explains Salinity



Spearman's Correlation

Calculating Spearman's Correlation

```
cor.test(salinity_prpc$mean_salinity,
         salinity_prpc$sum_prpc,
         method = "spearman")
```

Warning in cor.test.default(salinity_prpc\$mean_salinity,
salinity_prpc\$sum_prpc, : Cannot compute exact p-value with ties

Spearman's rank correlation rho

```
data: salinity_prpc$mean_salinity and salinity_prpc$sum_prpc
S = 939961, p-value = 0.8251
alternative hypothesis: true rho is not equal to 0
sample estimates:
      rho
0.01663311
```

Results

Spearman's rank correlation rho

data: salinity__prcpmean_salinityandsalinity_prcpsum__prcp S = 939961, p-value = 0.8251
alternative hypothesis: true rho is not equal to 0 sample estimates: rho 0.01663311

Interpreting the Results